Métodos Cuantitativos

Clase 06 - Distribuciones discretas

Unidad I: Estadística y Probabilidades con Python

Breve repaso

¿Qué veremos hoy?

- <u>**Variables aleatorias y distribuciones discretas**</u>
- Distribución uniforme discreta
- Distribución binomial
- PMF y CDF
- Distribución de Poisson

Variables aleatorias

Importancia de las Distribuciones

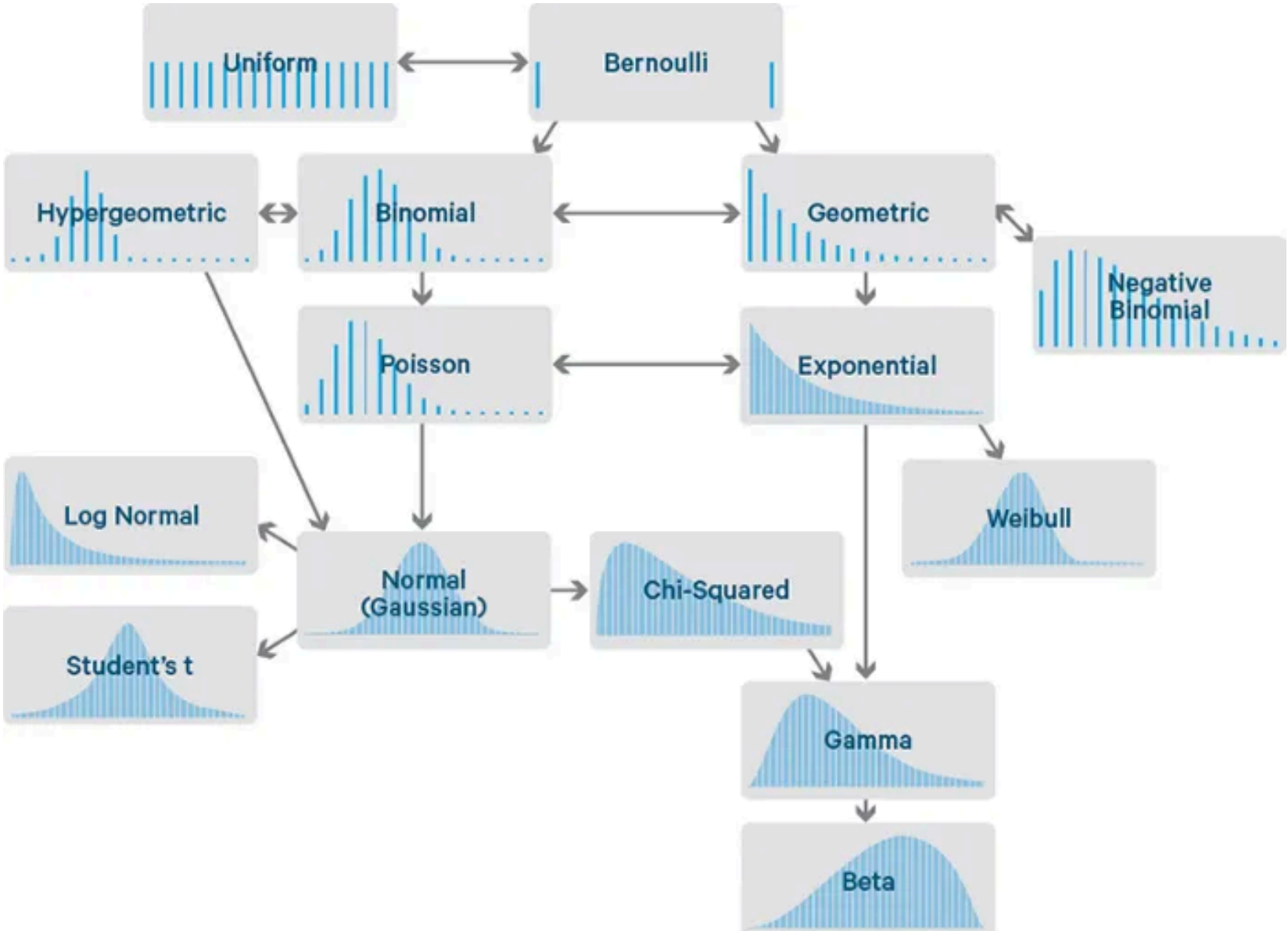
- Una **distribución de probabilidad** es una descripción que da la probabilidad para cada valor de la variable aleatoria.
- Las distribuciones de probabilidad son herramientas fundamentales en la estadística y la probabilidad, ya que nos permiten, modelar la aleatoriedad, realizar inferencias, predecir la probabilidad de que ocurran ciertos eventos, entre otros experimentos.

Distribuciones de probabilidad

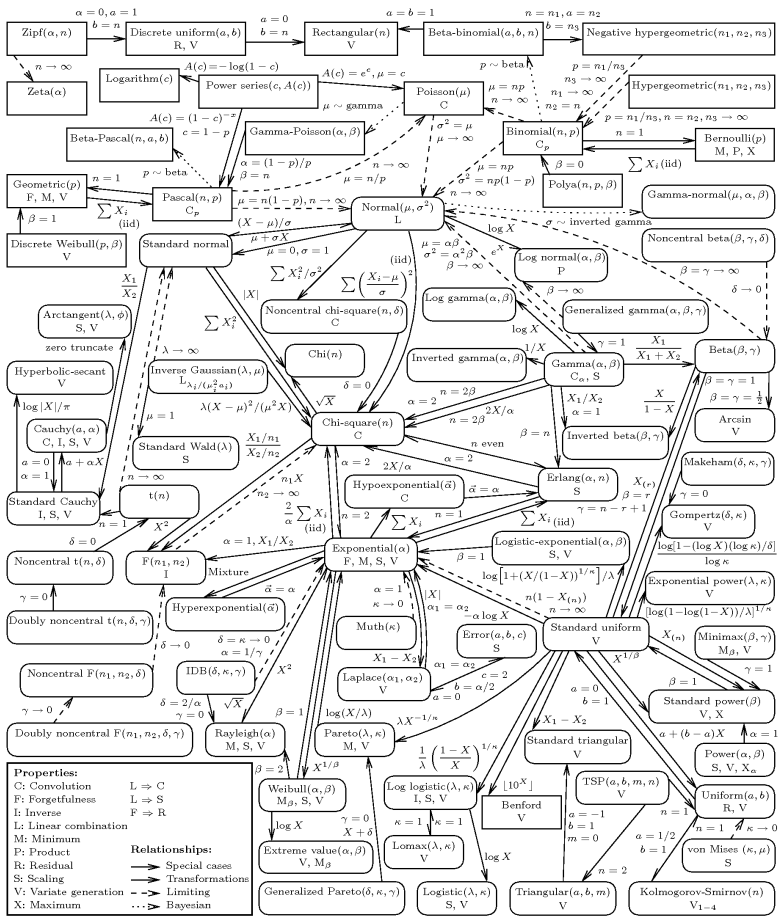
Cada distribución de probabilidad debe satisfacer los siguientes tres requisitos:

1. Hay una variable aleatoria <u>numérica</u> x, y sus valores numéricos están asociados con probabilidades correspondientes.
2. $\sum P(x) = 1$ donde x asume todos los valores posibles.
3. $0 \leq P(x) \leq 1$ para cada valor individual de la variable aleatoria x.

Distribuciones Discretas y Continuas



Distribuciones Discretas y Continuas



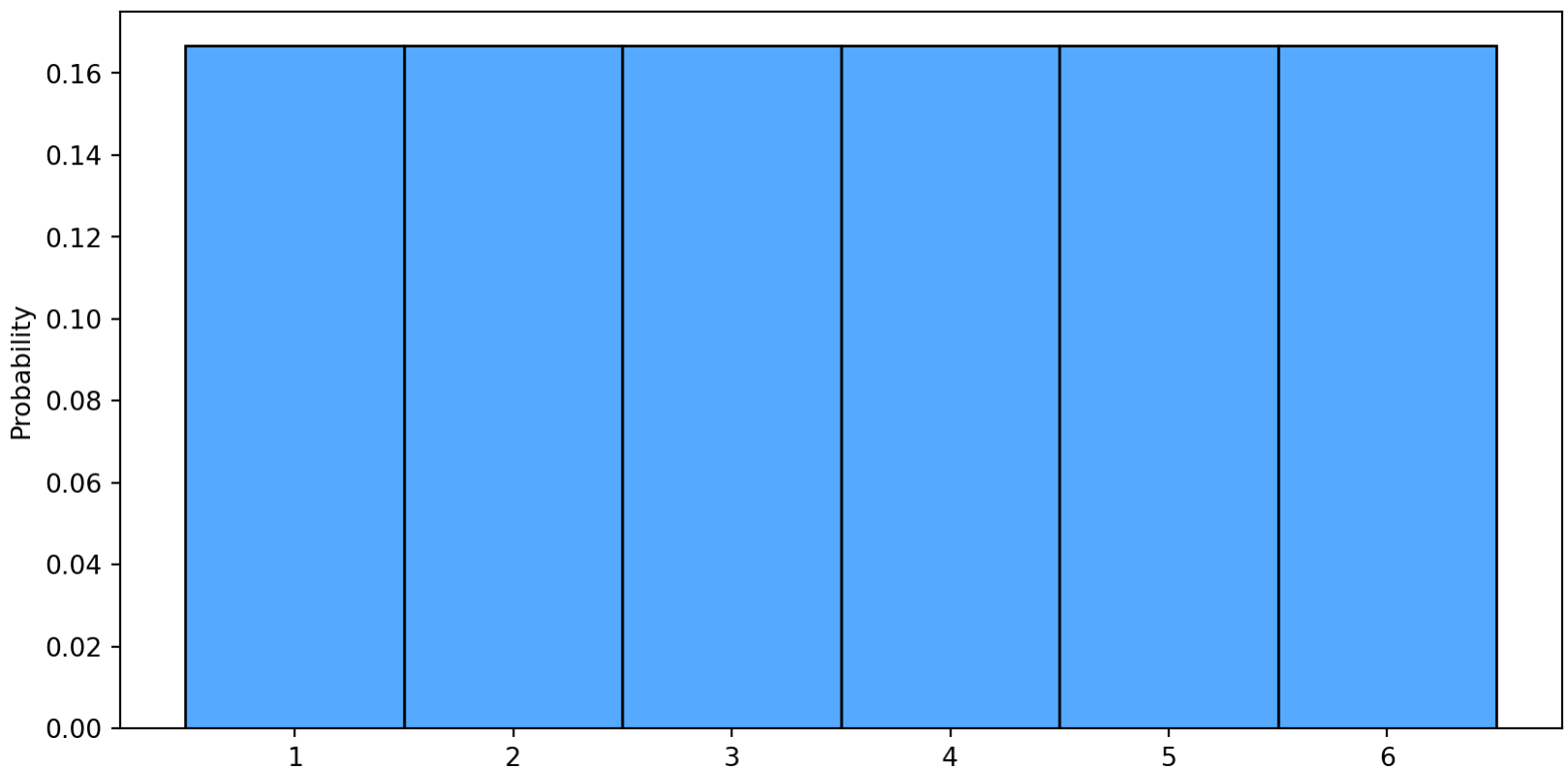
Distribuciones Discretas

Distribuciones Discretas

- Consideremos el evento de tirar un dado justo.
- Como aprendimos la semana anterior, este es un experimento equiprobable, es decir, hay 6 resultados posibles y cada uno tiene probabilidad $\frac{1}{6}$.
- Este es un ejemplo de una distribución de probabilidad, ya que se describe la probabilidad de cada posible resultado en el experimento.

Distribuciones Discretas

Si visualizáramos las probabilidades de cada resultado, obtendríamos lo siguiente:



Distribuciones Discretas

- Entonces, si quisiéramos, por ejemplo, calcular la probabilidad de obtener un número menor o igual a dos:
- $P(n \leq 2) : \frac{1}{6} + \frac{1}{6} = \frac{1}{3}$
- En este caso, como estamos <u>contando</u> los resultados del dado, se trata de una distribución **discreta**, y más específicamente, este tipo de distribución de probabilidad en la cual todos los resultados tienen la misma probabilidad se denomina **distribución discreta uniforme**.

¿Qué veremos hoy?

- Variables aleatorias y distribuciones discretas
- **Distribución uniforme discreta**
- Distribución binomial
- PMF y CDF
- Distribución de Poisson

Distribución Discreta Uniforme

- ¿Pero la distribución discreta uniforme teórica sucede en la realidad?
- Observemos la simulación de 500 tiradas de dados

Simulación de lanzamiento de dados 1 de 500
120
100
80
60
40
20
1
2
3
4
5
6
0:00 / 0:25

Distribución Discreta Uniforme

- El comportamiento empírico de la simulación se asemeja a la distribución, pero ¿cómo sabemos cuál debería ser el valor al qu deberíamos llegar para considerar que se está cumpliendo con distribución teórica?

- En este contexto, es importante introducir el concepto de <u>valor esperado de la distribución</u>, también conocido como **esperanza** el cual corresponde a la media de una distribución de probabilidad.

- En este caso, podemos obtener su resultado multiplicando cada valor por su probabilidad asociada:

- $(1 \cdot \frac{1}{6}) + (2 \cdot \frac{1}{6}) + (3 \cdot \frac{1}{6}) + (4 \cdot \frac{1}{6}) + (5 \cdot \frac{1}{6}) + (6 \cdot \frac{1}{6}) =$

Distribución Discreta Uniforme

- Pero, ¿cómo el valor esperado del resultado de un dado (denominado también con $\mathbf{E}$) puede ser 3.5 si un dado nunca podría dar ese resultado?
- Este resultado se debe a que el valor esperado es producto del promedio ponderado de todos los posibles resultados, donde cada resultado se multiplica por su probabilidad y luego se suman estos productos.
- Y aunque el resultado de un dado no puede ser exactamente 3.5, este valor representa el *"centro de masa"* de la distribución de probabilidad.

Distribución Discreta Uniforme

Si lanzamos el dado muchas veces y calculamos el promedio de todos los resultados, nos acercaremos cada vez más a 3.5 a medida que aumente el número de lanzamientos.

```python
import numpy as np
from scipy.stats import randint

sizes = [10, 100, 1_000, 10_000, 100_000]
for size in sizes:
    rolls = randint.rvs(1, 7, size=size)
    ev = np.mean(rolls)
    print(f"Tamaño: {size}, Esperanza: {ev}")
```

```
Tamaño: 10, Esperanza: 2.7
Tamaño: 100, Esperanza: 3.28
Tamaño: 1000, Esperanza: 3.535
Tamaño: 10000, Esperanza: 3.4861
Tamaño: 100000, Esperanza: 3.5047
```

Ley de los grandes números

- El comportamiento anteriormente observado corresponde a la **Ley de los grandes números**, la cual señala que a medida que se incrementa el tamaño de la muestra, la media de dicha muestra se acercará al valor esperado de la distribución.
- En otras palabras, a medida que realizamos más observaciones de un fenómeno aleatorio, la media de los resultados tiende a converger hacia el valor esperado teórico.

Distribuciones Discretas

- Volvamos a un ejemplo más sencillo: lanzar una moneda.
- Este experimento tiene dos posibles resultados: cara o cruz.
- Estos experimentos son llamados **experimentos de Bernoulli**, nombrados en honor a Jacob Bernoulli, quien fue uno de los primeros en estudiar este tipo de experimentos.
- En los experimentos de Bernoulli, solo pueden ocurrir dos resultados: **éxito** o **fracaso**.

Distribuciones Discretas

Para simular este tipo de experimentos en Python, usaremos la librería `scipy`.

```python
from scipy.stats import bernoulli

probabilidad_exito = 0.5
muestras_aleatorias = 1

tirada = bernoulli.rvs(p=probabilidad_exito, size=muestras_aleatorias)
print(tirada)
```

```
[0]
```

```python
muestras_aleatorias = 10
tirada = bernoulli.rvs(p=probabilidad_exito, size=muestras_aleatorias)
print(tirada)
print(f"n° caras: {sum(tirada)}")
```

```
[1 0 1 0 0 1 1 0 1 0]
n° caras: 5
```

¿Qué veremos hoy?

- Variables aleatorias y distribuciones discretas
- Distribución uniforme discreta
- **Distribución binomial**
- PMF y CDF
- Distribución de Poisson

Distribución Binomial

- El **número de éxitos** en una secuencia de experimentos Bernoulli independientes sigue una **distribución binomial**.
- Esta distribución describe la probabilidad de obtener un número específico de éxitos en una serie de ensayos independientes con solo dos resultados posibles: éxito o fracaso.
- Está definida por dos parámetros principales:
  - El número de ensayos.
  - La probabilidad de éxito en cada ensayo.

Distribución Binomial en Python

En Python, podemos simular este tipo de experimentos usando el módulo `binom` de `scipy`:

```python
from scipy.stats import binom

probabilidad_exito = 0.5
lanzamientos = 10
muestras_aleatorias = 1

resultado = binom.rvs(
  p=probabilidad_exito,
  n=lanzamientos,
  size=muestras_aleatorias
)
print(f"De {lanzamientos} tiradas, se obtuvieron {resultado[0]} caras")
```

```
De 10 tiradas, se obtuvieron 4 caras
```

Distribución Binomial en Python

Múltiples muestras aleatorias:

```
lanzamientos = 10
muestras_aleatorias = 10
binom.rvs(p=probabilidad_exito, n=lanzamientos, size=muestras_aleatorias)
```

```
array([5, 4, 4, 5, 5, 4, 6, 6, 4, 8])
```

Simular moneda trucada:

```
probabilidad_exito = 0.3
binom.rvs(p=probabilidad_exito, n=lanzamientos, size=muestras_aleatorias)
```

```
array([1, 2, 3, 4, 3, 2, 1, 3, 3, 6])
```

Distribución Binomial en Python

Cada vez que llevamos a cabo una simulación, estamos generando números pseudoaleatorios, lo que significa que obtenemos resultados diferentes con cada ejecución del experimento. Para contrarrestar esta variabilidad, podemos utilizar lo que se conoce como una **semilla**.

```
binom.rvs(p=0.5, n=10, size=10)
```

```
array([6, 5, 2, 3, 5, 5, 5, 5, 6, 4])
```

Semilla con **scipy**:

```
binom.rvs(p=0.5, n=10, size=10, random_state=2024)
```

```
array([5, 6, 4, 2, 4, 3, 6, 6, 5, 5])
```

Semilla con **numpy**:

```
np.random.seed(2024)
binom.rvs(p=0.5, n=10, size=10)
```

¿Qué veremos hoy?

- Variables aleatorias y distribuciones discretas
- Distribución uniforme discreta
- Distribución binomial
- **PMF y CDF**
- Distribución de Poisson

Funciones de masa de probabilidad y distribución

- Las funciones de masa de probabilidad y las funciones de distribución son herramientas fundamentales en estadística y teoría de la probabilidad.
- La **función de masa de probabilidad** describe la probabilidad de que una variable aleatoria discreta tome un valor específico.
- Por otro lado, la **función de distribución acumulativa** proporciona la probabilidad de que una variable aleatoria discreta sea menor o igual a un cierto valor.

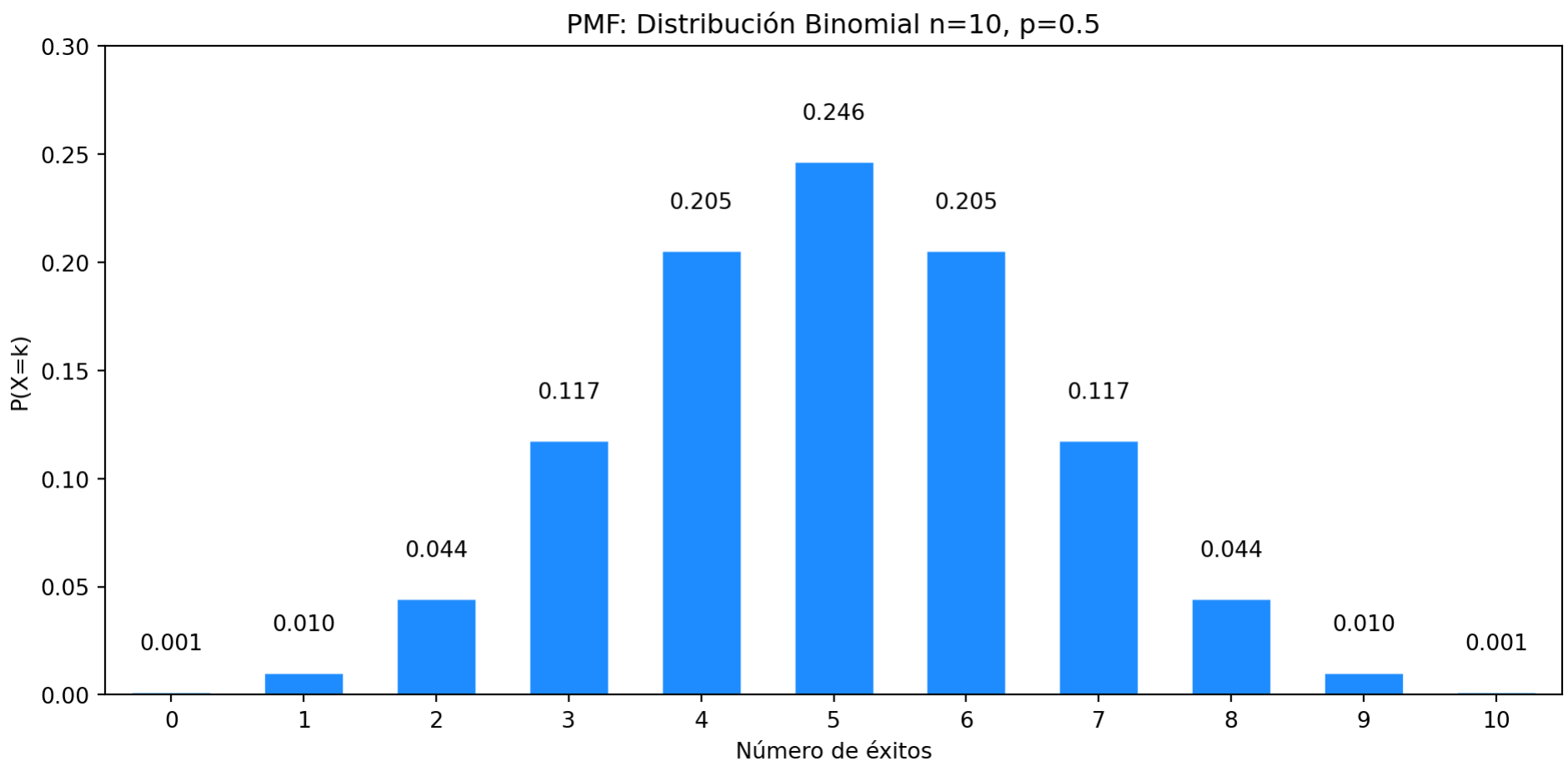
PMF: Distribución Binomial

La función de masa de probabilidad de la distribución binomial (PMF) se puede expresar como:

$$P(X = k) = \binom{n}{k} \cdot p^k \cdot (1 - p)^{n-k}$$

Donde:

PMF: Distribución Binomial



PMF: Distribución Binomial

Para calcular la función de masa de probabilidad en Python, podemos usar la librería `scipy`, siguiendo la sintaxis:

```
binom.pmf(k, n, p)
```

Por ejemplo:

```
# probabilidad de obtener 2 caras de 10 lanzamientos de monedas justas
binom.pmf(k=2, n=10, p=0.5)
```

```
np.float64(0.043945312500000076)
```

```
# probabilidad de obtener 5 caras de 10 lanzamientos de monedas justas
binom.pmf(k=5, n=10, p=0.5)
```

```
np.float64(0.2460937500000002)
```

PMF: Distribución Binomial

¿Qué creen que sucede si **aumentamos proporcionalmente** el k y n?

Veamos que sucede…

```
# prob. de obtener 25 caras de 50 lanzamientos de monedas justas
binom.pmf(k=25, n=50, p=0.5)
```

```
np.float64(0.11227517265921719)
```

```
# prob. de obtener 50 caras de 100 lanzamientos de monedas justas
binom.pmf(k=50, n=100, p=0.5)
```

```
np.float64(0.07958923738717871)
```

```
# prob. de obtener 50 caras de 100 lanzamientos de monedas a favor nuestro
binom.pmf(k=50, n=100, p=0.7)
```

```
np.float64(1.3026227131445535e-05)
```

CDF: Distribución Binomial

Los resultados anteriores resultaban en escenarios de baja probabilidad ya que estábamos calculando la probabilidad exacta de obtener k caras de n lanzamientos.

En cambio, podemos calcular la probabilidad de obtener k caras o menos.

Este cálculo lo realizamos usando la **función de distribución acumulativa** o **CDF** por sus siglas en inglés.

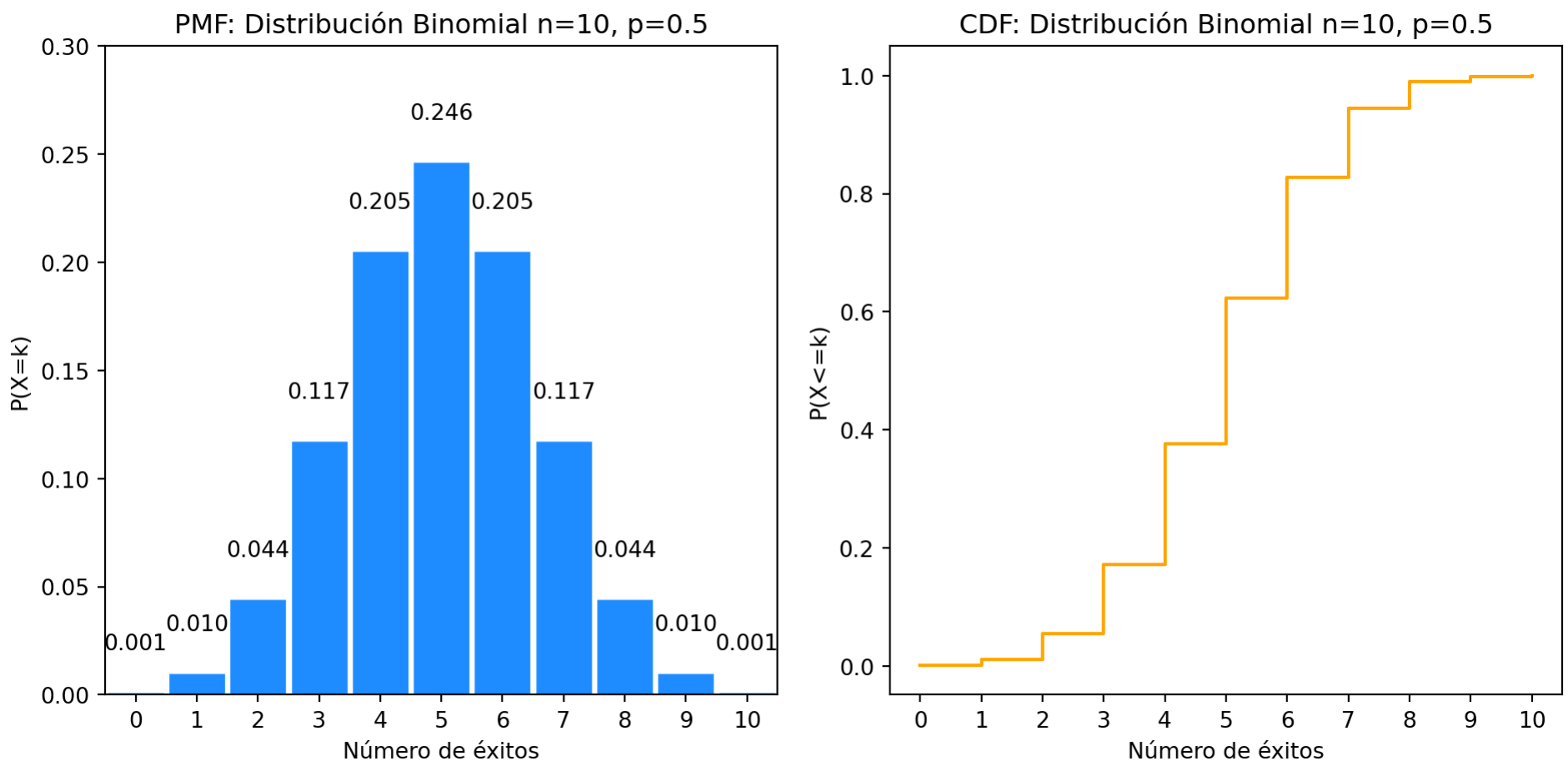
CDF: Distribución Binomial

La función de distribución acumulativa de la distribución binomial se puede expresar como:

$$F(k; n, p) = \sum_{i=0}^{k} \binom{n}{i} \cdot p^{i} \cdot (1-p)^{n-i}$$

Donde:

CDF: Distribución Binomial



CDF: Distribución Binomial

Para calcular la función de distribución acumulativa en Python, podemos usar la librería `scipy`, siguiendo la sintaxis:

```
binom.cdf(k, n, p)
```

Por ejemplo:

```
# prob. de obtener 2 caras o menos de 10 lanzamientos de monedas justas
binom.cdf(k=2, n=10, p=0.5)
```

```
np.float64(0.0546875)
```

```
# prob. de obtener 5 caras o menos de 10 lanzamientos de monedas justas
binom.cdf(k=5, n=10, p=0.5)
```

```
np.float64(0.623046875)
```

```
# prob. de obtener 25 caras o menos de 50 lanzamientos de monedas justas
binom.cdf(k=25, n=50, p=0.5)
```

```
np.float64(0.5561375863296085)
```

```
# prob. de obtener 50 caras o menos de 100 lanzamientos de monedas justas
binom.cdf(k=50, n=100, p=0.5)
```

```
np.float64(0.5397946186935896)
```

CDF: Distribución Binomial

También podemos calcular la probabilidad de obtener k o más caras después de n lanzamientos, esto es llamado el **complemento**, y se puede obtener de la siguiente manera:

```
# prob. de más de 30 caras de 100 lanzamientos de monedas justas
1- binom.cdf(k=30, n=100, p=0.5)
```

```
np.float64(0.999960749301772)
```

```
# prob. de más de 30 caras de 100 lanzamientos de monedas justas
binom.sf(k=30, n=100, p=0.5)
```

```
np.float64(0.999960749301772)
```

¿Qué veremos hoy?

- Variables aleatorias y distribuciones discretas
- Distribución uniforme discreta
- Distribución binomial
- PMF y CDF
- **Distribución de Poisson**

Distribución de Poisson

Distribución de Poisson

- La distribución de Poisson es una **distribución de probabilidad discreta** que se utiliza comúnmente para modelar el número de eventos que ocurren en un intervalo de tiempo o espacio fijo, dado un promedio conocido de eventos por intervalo.
- Esta distribución es útil en una variedad de aplicaciones, como en el modelamiento de la frecuencia de llegada de llamadas telefónicas, errores de impresión en un documento, incidentes de tráfico, etc.

Distribución de Poisson en Python

En Python, podemos simular este tipo de experimentos usando el módulo `poisson` de `scipy`:

```
from scipy.stats import poisson

lambda_ = 15

resultado = poisson.rvs(lambda_, size=10)
print(resultado)
```

```
[17 14 17 12 25 12 19 14 12 12]
```

```
from scipy.stats import poisson

lambda_ = 20

resultado = poisson.rvs(lambda_, size=20)
print(resultado)
```

```
[26 24 19 19 21 27 20 20 16 18 22 21 25 20 17 19 16 25 20 24]
```

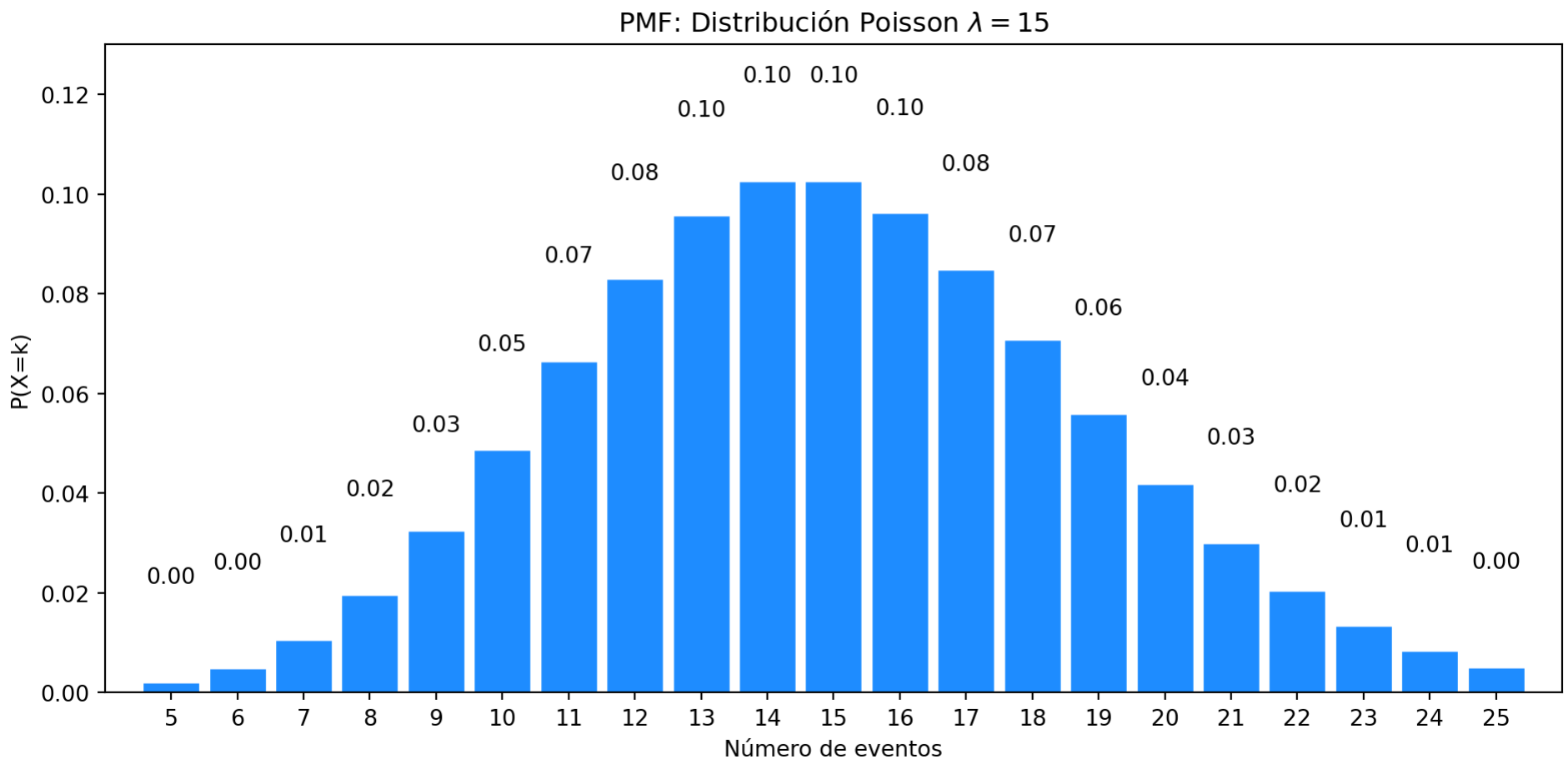
PMF: Distribución de Poisson

La distribución de Poisson se caracteriza por un solo parámetro λ, que representa la tasa promedio de ocurrencia de eventos en el intervalo considerado. La función de masa de probabilidad (PMF) de la distribución de Poisson se define como:

$$P(X = k) = \frac{\lambda^k \cdot e^{-\lambda}}{k!}$$

Donde:

PMF: Distribución de Poisson



PMF: Distribución de Poisson

Para calcular la función de masa de probabilidad en Python, podemos usar la librería `scipy`, siguiendo la sintaxis:

```
poisson.pmf(k, mu)
```

Por ejemplo:

```
# probabilidad de obtener 10 eventos para un lambda de 15
poisson.pmf(10, 15)
```

```
np.float64(0.04861075082960534)
```

```
# probabilidad de obtener 15 eventos para un lambda de 15
poisson.pmf(15, 15)
```

```
np.float64(0.10243586666453426)
```

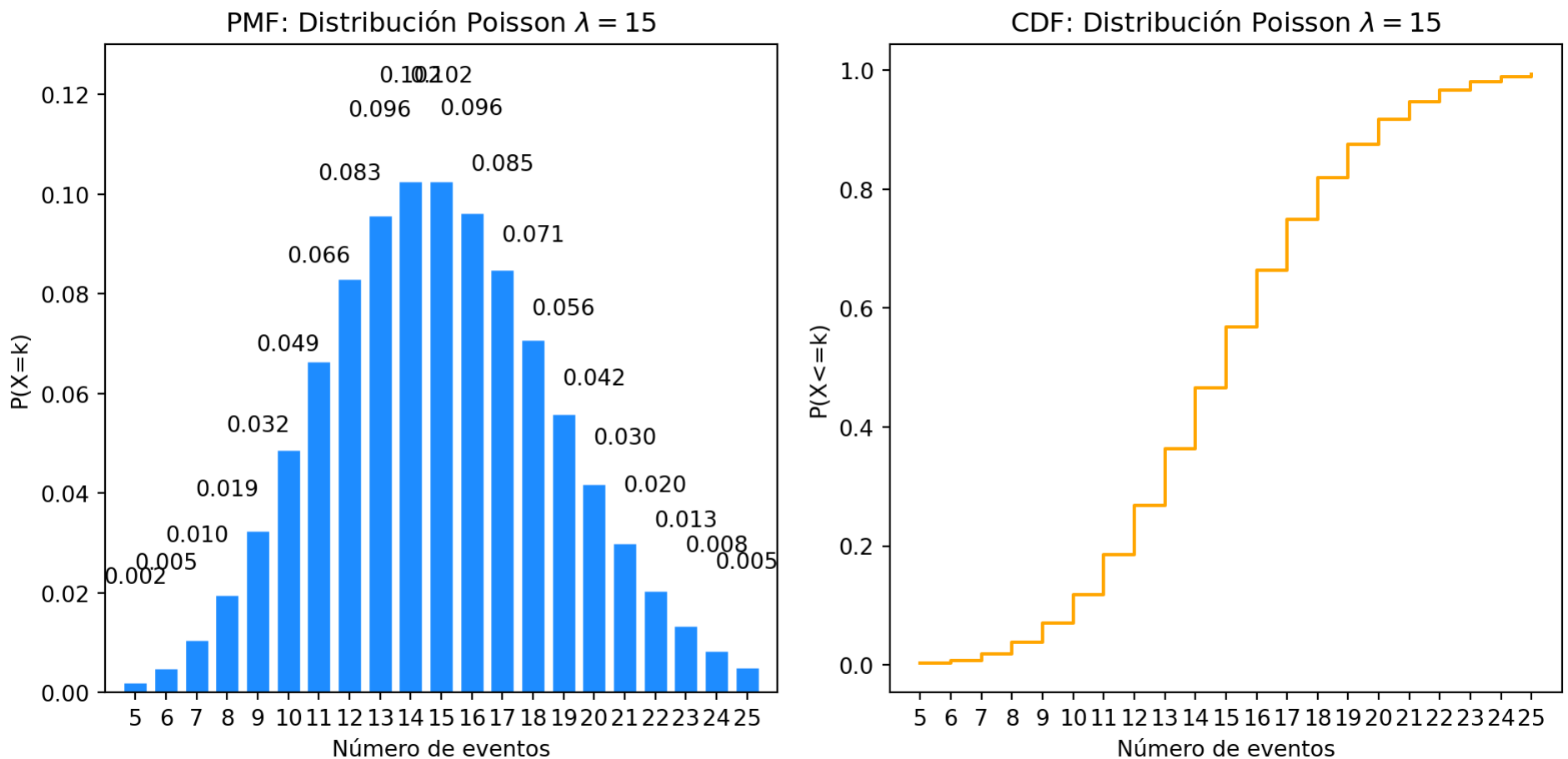
CDF: Distribución de Poisson

La función de distribución acumulativa de la distribución Poisson se puede expresar como:

$$F(k;\lambda) = \sum_{i=0}^{k} \frac{e^{-\lambda} \cdot \lambda^{i}}{i!}$$

Donde:

CDF: Distribución de Poisson



CDF: Distribución de Poisson

Para calcular la función de distribución acumulativa en Python, podemos usar la librería `scipy`, siguiendo la sintaxis:

```
poisson.cdf(k, mu)
```

Por ejemplo:

```
# prob. de obtener 10 eventos o menos para un lambda de 15
poisson.cdf(10, 15)
```

```
np.float64(0.11846441152901505)
```

```
# prob. de obtener 15 eventos o menos para un lambda de 15
poisson.cdf(15, 15)
```

```
np.float64(0.5680895756085438)
```

```
# prob. de obtener 20 eventos o menos para un lambda de 15
poisson.cdf(20, 15)
```

```
np.float64(0.9170290899685398)
```

CDF: Distribución de Poisson

Calcular el **complemento**:

```
poisson.sf(k, mu)
```

```
# prob. de más de 20 o más eventos para un lambda de 15
1- poisson.cdf(20, 15)
```

```
np.float64(0.08297091003146018)
```

```
# prob. de más de 20 o más eventos para un lambda de 15
poisson.sf(20, 15)
```

```
np.float64(0.08297091003146019)
```

Bibliografía recomendada

- Estadística, Capítulo 5, Triola Mario F, 2018 12a edición.
- Introducción a la Estadística, Capítulo 5, Sheldon M. Ross, 2014.